Vani Integration — Phase 0 Answers (from Deepak)

For: IoT Platform 2.0 team · **Re:** Vani Voice Integration Plan, Phase 0 — Agree the contract (steps 1–3) and the "Open before we start" items **Date:** 5 Nov 2026 · **Vani base URL:** https://vani-service-production.up.railway.app · **Caller ID:** +1 817-809-3797

1. Platform token — exists, proven, shared out-of-band

- The token **already exists and is live-proven**: the 3 Nov acceptance test fired `POST /platform/alert-call` with it and got `202 dialed` (that call rang Deepak at 22:18 CT).
- On Vani's side it is the Railway variable **`VANI_PLATFORM_TOKEN`** on the **vani-service** service (deliberately the same name you will use — one shared secret, set identically on both services).
- **Handover:** Deepak copies it from Railway (vani-service → Variables) and shares it with the team lead **out-of-band** — never in this document, a ticket, chat or code. You store it as the `VANI_PLATFORM_TOKEN` secret on your API service.
- Until it is set on your side, nothing is sent and requests wait without consuming retries — as your plan already handles.
- **Rotation rule:** if it is ever rotated, both services change in the same sitting; until they match, triggers return 401 and stop (your handling already records and stops — correct).

2. Severity mapping — DECIDED: platform **`high`** → Vani **`warning`**

Record in `docs/ai/DECISIONS.md`:

Platform 2.0	Vani	Behaviour
`critical`	`critical`	Calls immediately, **including quiet hours**
`high`	**`warning`**	**Calls in business hours; deferred 8 PM–8 AM customer time**
`medium`, `low`	`warning`	Business hours
`none`	`info`	Never calls; outcome callback still fires

Rationale: `critical` is the platform's night tier — that is what it is for. `high` means "act within the week" (your Phase 4 bearing case is a *critical* rule, which is the right pattern); waking a customer at night is reserved for critical. If a class of `high` alerts ever needs night calling, the fix is to raise it as `critical` in the platform, not to remap the tier. (`VANI_HIGH_SEVERITY` can stay unset — your coded default already sends `high` as `warning`.) Vani's calling policy stands as documented: up to 2 attempts 45 minutes apart, voicemail on the final attempt, caps of 3 calls/machine/day and 6/account/day, per-account opt-out.

3. Callback URL and secret — AGREED, with one small Vani-side change

- **URL shape agreed:** `https://<api-host>/api/v1/vani/outcome-callback`. Send Deepak the concrete URL once the API host is final; he will store it on the `usa-test-01` registration (the registration's `callback_url` is blank today — acceptance item 1 — and is set before the Phase 5 live test, not needed for Phases 1–4).
- **Authentication agreed:** shared secret in the **`X-Vani-Secret`** header, checked against your `VANI_CALLBACK_SECRET`.
- **Who creates the secret:** the platform team generates it and shares it with Deepak out-of-band (same handling as the token). **Vani-side change accepted on our side:** today Vani's outcome callback sends no header (verified in the live code, `post_outcome_callback`); we will add the secret — read from a `VANI_CALLBACK_SECRET` variable on vani-service — as the `X-Vani-Secret` header on every outcome callback before the live test. Until both sides hold it, your endpoint refuses callbacks (503) and Vani's ledger still records every outcome, so nothing is lost while it is unset.
- Dedupe on (`event_id`, `attempt`) and the five-value outcome enum stand as in the Nov 3 guide.

4. Railway variables — written approval (the narrow exception)

Approved: adding **`VANI_PLATFORM_TOKEN`**, **`VANI_BASE_URL`** (= `https://vani-service-production.up.railway.app`), **`VANI_CALLBACK_SECRET`** and **`VANI_CALLS_ENABLED`** to the existing API service — this approval covers these four variables on that service only. `VANI_CALLS_ENABLED` stays `false` until the Phase 5 live test is booked with Deepak.

5. Companion spec — found, in the handoff bundle with this document

`ThingsAlive_Vani_MachineLane_Spec_20261103.md` exists on our side and is **in the handoff bundle** (4 documents + bundle README), together with the *Vani × IoT Platform Integration Guide, Nov 3 2026* it supports. **How to get them:** the bundle accompanies this document — if you received this file alone, ask Deepak for the bundle; recommended: commit the bundle copies into

your repo (`docs/ai/vani/`) so the references resolve for the whole team. The plan's reading is right that live behaviour wins where they differ; the spec's §3 is the contract the live service was built and tested against (registry shape §3.3, route rules §3.2, callback §3.4). The bundle README lists every document referenced anywhere in these answers and where it lives — the only referenced item deliberately not in the bundle is the platform token (§1, out-of-band by design).

6. Scope — ADOPTED 5 Nov 2026 (contract addition to this Phase 0)

Deepak has reviewed the plan's scope against the Nov 3 machine-lane vision and **adopted the four-lane scope** (full memo: `VANI_INTEGRATION_SCOPE_REVIEW.md`, in the handoff bundle with this document). What it means for the contract:

- **The integration is not alerts-only.** Four lanes: **A · Alerts** (your Phases 0–5, unchanged, first to go live) · **B · Predictions & trigger actions** (restore `trigger.kind = trigger_action`; Live Predictions and health-band crossings may fire triggers on the same endpoint, same idempotency, same severity policy; `confidence` becomes its own packet field) · **C · Inbound full review** · **D · Operation loop**.
- **Phase 6 is APPROVED as built.** The ask-the-platform endpoint (one question tool, caller-scoped, answer-only) is agreed — it no longer waits. Sequencing stays yours: finish Phase 5 and the live acceptance test first. Fact for the build: Vani can pass the caller's number (E.164) and a `conversation_id` with each question, as your `/api/v1/vani/ask` contract expects. Whether Vani-registered numbers and 2.0 users are the same people is settled per account at registration, starting with the test account (whose registered number is its admin's).
- **Informs vs acts — DECIDED: acts (Lane D).** Extend the outcome callback with a **`customer_decision`** field (e.g. `inspection_approved`, `call_back_requested`, `declined`, `acknowledged`) so the platform can act on what the customer said: acknowledge the alert, create the work order, schedule the follow-up. Vani acts **only on the customer's explicit words on the call, confirmed back by Vani before she closes** — never on inference.
- **Admin panel — per-customer FEATURE SELECTION (new requirement).** In the Vani Admin panel, every customer account carries feature toggles — `features: { alerts, health, predictions, operations }` on the registration (defaults: alerts ON, the rest OFF until Deepak sets them; global defaults in the Rules editor, per-account selection overrides). The toggles scope outbound triggers **and** inbound discussion. The route-rules engine checks the feature **first**: a trigger whose feature is OFF for the account is accepted, logged and reported as `suppressed_by_policy` with reason **`feature_not_enabled`**. Snapshots keep flowing for all registered machines regardless of toggles. Acceptance addition: with `predictions` OFF on `usa-test-01`, a prediction trigger is suppressed with that reason while an alert trigger still dials; toggle it ON and the same prediction trigger dials.
- Everything else in §§1–5 stands: token handover, `high → warning`, callback URL + `X-Vani-Secret`, the Railway variables, and the calling policy (attempts, caps, quiet hours) apply identically across all four lanes.

— Deepak Panguluri (via Dippy) · Questions on the contract come back to Deepak; calling policy is his and is administered in the Vani Admin tab.